Time Series

1. 🌟 Linear Regression With Time Series

- Forecast = Predict
- ARIMA : Modèle de prédiction des séries temporelles

- Le retail est un terme qui englobe tous les secteurs du commerce de détail en ligne.

- There are two kinds of features unique to time series: **time-step features** and **lag features**.

- Time-step features are features we can derive directly from the time index. The most basic time-step feature is the **time dummy**, which counts off time steps in the series from beginning to end.

- To make a **lag feature** we shift the observations of the target series so that they appear to have occured later in time. Here we've created a 1-step lag feature, though shifting by multiple steps is possible too.

- The best time series models will usually include some combination of time-step features and lag features.

2. 🌟 Trend

- The **trend** component of a time series represents a persistent, long-term change in the mean of the series. The trend is the slowest-moving part of a series, the part representing the largest time scale of importance.

- For a change to be a part of the trend, it should occur over a longer period than any seasonal changes. To visualize a trend, therefore, we take an average over a period longer than any seasonal period in the series.

- To create a moving average, first use the rolling method to begin a windowed computation. Follow this by the mean method to compute the average over the window.

- ***If you've worked with economic time series before, you might guess that the growth rate in Food and Beverage Sales is best expressed as a percent change. Percent change can often be modeled using an exponential curve. (Don't worry if this isn't familiar!)*** : lorsqu'on analyse les séries temporelles économiques, notamment les ventes dans le secteur de l'alimentation et des boissons, il est courant d'exprimer le taux de croissance en pourcentage pour mieux comprendre l'évolution. Cette mesure de pourcentage peut souvent être représentée par une courbe exponentielle, une forme mathématique qui décrit une croissance où le taux de changement est proportionnel à la valeur actuelle.

3. 🌟 Seasonality

- We say that a time series exhibits **seasonality** whenever there is a regular, periodic change in the mean of the series.

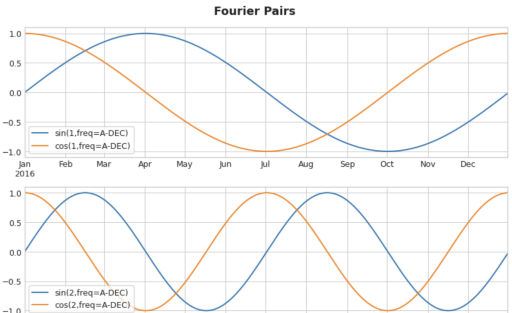
- We will learn two kinds of features that model seasonality. The first kind, **indicators**, is best for a season with few observations, like a weekly season of daily observations. The second kind, **Fourier features**, is best for a season with many observations, like an annual season of daily observations.

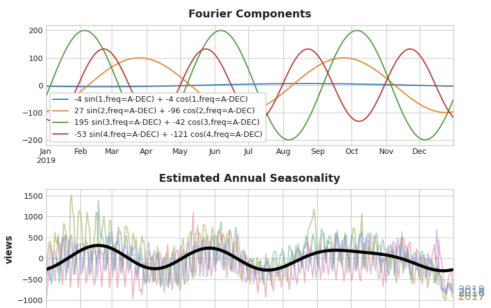
- **Seasonal indicators** are binary features that represent seasonal differences in the level of a time series. Seasonal indicators are what you get if you treat a seasonal period as a categorical feature and apply one-hot encoding.

- **Fourier features** are better suited for long seasons over **many observations** where indicators would be impractical. Instead of creating a feature for each date, Fourier features try to capture the overall shape of the seasonal curve with just a few features.

- The idea is to include in our training data **periodic curves having the same frequencies as the season we are trying to model**. The curves we use are those of the trigonometric functions sine and cosine.

- **Fourier features** are pairs of sine and cosine curves ⇒ If we add a set of these sine / cosine curves to our training data, the linear regression algorithm will figure out **the weights that will fit the seasonal component in the target series.**





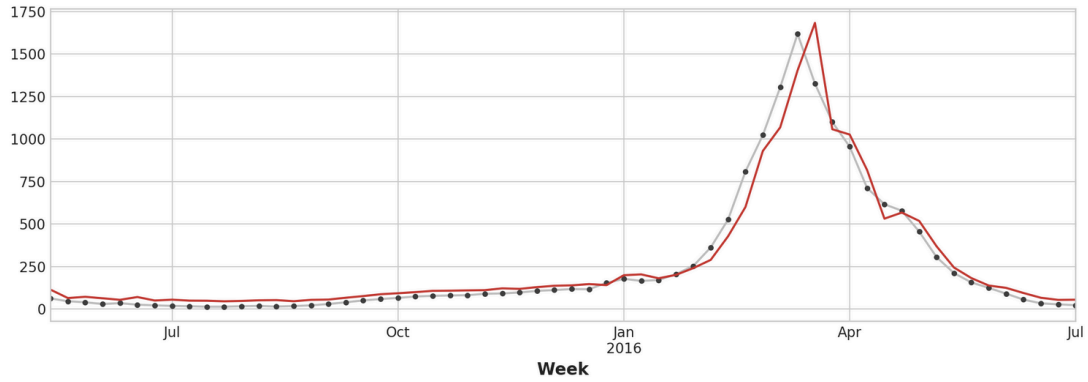
- How many Fourier pairs should we actually include in our feature set? We can answer this question with the **periodogram(?)**.

4. 🌟 Time Series as Features

- Some time series properties, however, can only be modeled as *serially dependent* properties, that is, using as features past values of the target series.
- **Cycles** are patterns of growth and decay in a time series associated with how the **value in a series at one time depends on values at previous times**, but not necessarily on the time step itself.
- **Cyclic behavior** is characteristic of systems that can affect themselves or whose reactions persist over time.
- What distinguishes cyclic behavior from seasonality is that cycles are not necessarily time dependent, as seasons are.
- What happens in a cycle is less about the particular date of occurrence, and more about **what has happened in the recent past**.
- To investigate possible serial dependence (like cycles) in a time series, we need to create "**lagged**" copies of the series.
- By lagging a time series, we can make its past values appear contemporaneous with the values we are trying to predict (in the same row, in other words). **This makes lagged series useful as features for modeling serial dependence.**
- A **lag plot** of a time series shows its values plotted against its lags. Serial dependence in a time series will often become apparent by looking at a lag plot.
- Plotting the partial autocorrelation can help you choose which lag features to use(the amount of "new" correlation the lag contributes).
- **The partial autocorrelations suggest the dependence can be captured using lags different than 0.**
- We need to be mindful that autocorrelation and partial autocorrelation are measures of *linear* dependence. Because real-world time series often have substantial non-linear dependences, it's best to look at a lag plot (or use some more general measure of dependence, like mutual information) when choosing lag features. The *Sunspots* series has lags with non-linear dependence which we might overlook with autocorrelation.
- In previous lessons, we were able to create forecasts for as many steps as we liked beyond the training data. When using lag features, however, we are limited to forecasting time steps whose lagged values are available.
- We can lag a time series in Pandas with the `shift` **method.** For this problem, we'll fill in the missing values the lagging creates with `0.0`.

Looking just at the forecast values, we can see how our model needs a time step to react to sudden changes in the target series. This is a common limitation of models using only lags of the target series as features.

Show hidden code



- To improve the forecast we could try to find **leading indicators**, time series that could provide an "early warning" for changes in flu cases. For our approach then we'll add to our training data the popularity of some flu-related search terms as measured by Google Trends.
- In the context of search terms, **popularity** typically refers to the frequency or volume of searches for a particular term on a search engine. For example, a search term with high popularity would be one that is frequently searched for by many people.

5. 🌟 Hybrid Models

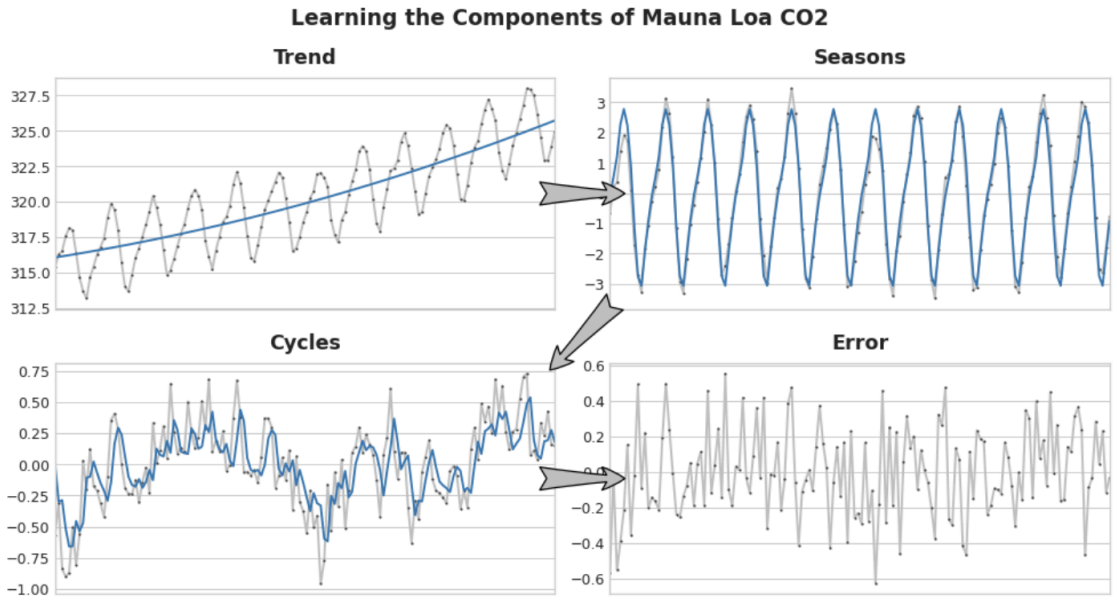
- So that we can design effective hybrids, we need a better understanding of how time series are constructed. We've studied up to now three patterns of dependence: **trend, seasons,** and **cycles.**
- Many time series can be closely described by an additive model of just these three components plus some essentially unpredictable, entirely random *error*:

series = trend + seasons + cycles + error

- The **residuals** of a model are the difference between the target the model was trained on and the predictions the model makes -- the difference between the actual curve and the fitted curve, in other words.

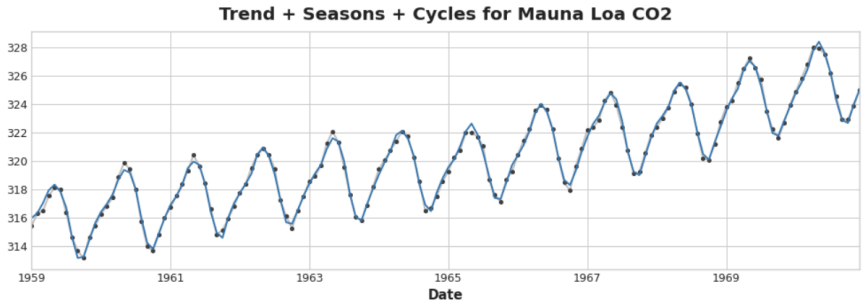
- We could imagine learning the components of a time series as an iterative process:
    - first learn the trend and subtract it out from the series,

- then learn the seasonality from the detrended residuals and subtract the seasons out,
- then learn the cycles and subtract the cycles out,
- and finally only the unpredictable error remains.



Learning the components of Mauna Loa CO2 step by step. Subtract the fitted curve (blue) from its series to get the series in the next step.

- Add together all the components we learned and we get the complete model. This is essentially what linear regression would do if you trained it on a complete set of features modeling trend, seasons, and cycles.



Add the learned components to get a complete model.

- In previous lessons, we used a single algorithm (linear regression) to learn all the components at once. But it's also possible to use one algorithm for some of the components and another algorithm for the rest. This way we can always choose the best algorithm for each component.

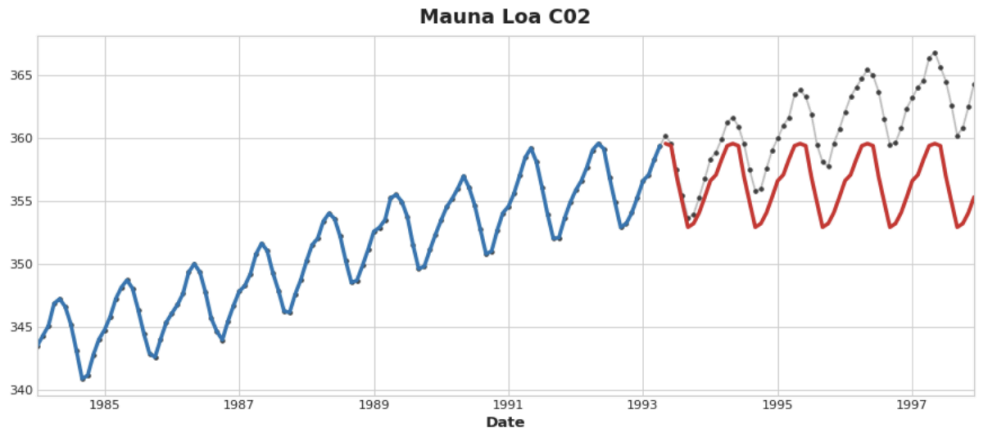
- To do this, we use one algorithm to fit the original series and then the second algorithm to fit the residual series.

- While it's possible to use more than two models, in practice it doesn't seem to be especially helpful. In fact, the most common strategy for constructing hybrids is the one we've just described: **a simple (usually linear) learning algorithm** followed by **a complex, non-linear learner like GBDTs or a deep neural net**.

- The simple models are typically designed as a "helper" for the powerful algorithm that follows.

- There are many ways you could combine machine learning models besides the way we've outlined in this lesson.
- There are generally two ways a regression algorithm can make predictions:
    - **Transforming the features** : Feature-transforming algorithms learn some mathematical function that takes features as an input and then combines and transforms them to produce an output that matches the target values in the training set. Linear regression and neural nets are of this kind.
    - **Transforming the target** : Target-transforming algorithms use the features to group the target values in the training set and make predictions by averaging values in a group; a set of features just indicates which group to average. Decision trees and nearest neighbors are of this kind.

- ⚠️**The important thing is this:** feature transformers generally can **extrapolate** target values beyond the training set given appropriate features as inputs, but the predictions of target transformers will always be bound within the range of the training set ⇒ If the time dummy continues counting time steps, linear regression continues drawing the trend line. Given the same time dummy, a decision tree will predict the trend indicated by the last step of the training data into the future forever. *Decision trees cannot extrapolate trends.* Random forests and gradient boosted decision trees (like XGBoost) are ensembles of decision trees, so they also cannot extrapolate trends.



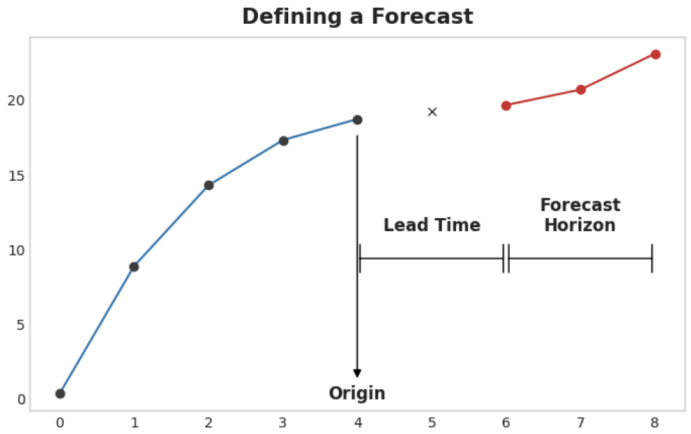
A decision tree will fail to extrapolate a trend beyond the training set.

- This difference is what motivates the hybrid design in this lesson: use linear regression to extrapolate the trend, transform the *target* to remove the trend, and apply XGBoost to the detrended residuals ⇒ **boosted hybrids.**

- To hybridize a neural net (a feature transformer), you could instead **include the predictions of another model as a feature, which the neural net would then include as part of its own predictions** ⇒ **stacked hybrids.**
- The method of fitting to residuals is actually the same method the gradient boosting algorithm uses, so we will call these **boosted hybrids.**
- The method of using predictions as features is known as "stacking", so we will call these **stacked hybrids.**

6. 🌟 Forecasting With Machine Learning

- There are two things to establish before designing a forecasting model:
  - what information is available at the time a forecast is made (**features**).
  - the time period during which you require forecasted values (**target**).
- **The forecast origin** is the time at which you are making a forecast. Practically, you might consider the forecast origin to be the last time for which you have training data for the time being predicted. Everything up to the origin can be used to create features.
- **The forecast horizon** is the time for which you are making a forecast. We often describe a forecast by the number of time steps in its horizon: a "1-step" forecast or "5-step" forecast, say. The forecast horizon describes the target.
- **A forecast's lead time** is described by the number of steps from origin to horizon: a "1-step ahead" or "3-step ahead" forecast, say. In practice, it may be necessary for a forecast to begin multiple steps ahead of the origin because of **delays in data acquisition or processing**.

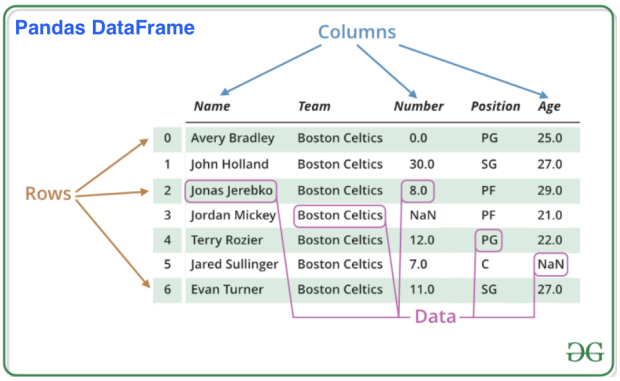


A three-step forecast horizon with a two-step lead time, using four lag features. The figure represents what would be a single row of training data -- data for a single prediction, in other words.

- In order to forecast time series with ML algorithms, we start with preparing data and for this we need to **transform the series into a dataframe** we can use with those

algorithms. (Unless, of course, you are only using deterministic features like trend and seasonality.)

A DataFrame is a two-dimensional data structure, that is, the data is tabularly aligned in rows and columns.

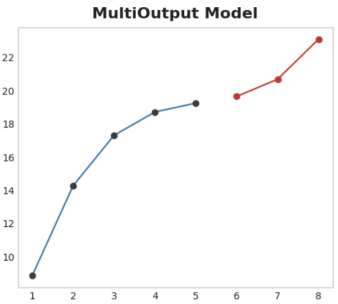


For more details check :

- https://www.kaggle.com/code/arnopub/pandas-pr-sentation-des-dataframe
- https://www.w3schools.com/python/pandas/pandas_dataframes.asp

- Each row in a dataframe represents a single forecast. The time index of the row is the first time in the forecast horizon, but we arrange values for the entire horizon in the same row. For multistep forecasts, this means we are requiring a model to produce multiple outputs, one for each step.

- There are a number of strategies for **producing the multiple target steps required for a forecast**. We'll outline four common strategies, each with strengths and weaknesses.
  - 1**Multioutput model:** Use a model that produces multiple outputs naturally. Linear regression and neural networks can both produce multiple outputs. This strategy is simple and efficient, 👎 **but not possible for every algorithm you might want to use. XGBoost can't do this, for instance.**

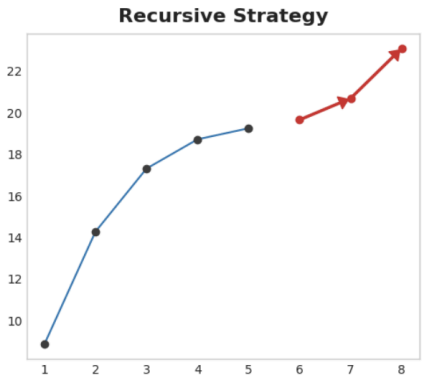


  - 2**Direct strategy:** Train a separate model for each step in the horizon: one model forecasts 1-step ahead, another 2-steps ahead, and so on. Forecasting 1-step ahead is a different problem than 2-steps ahead (and so on), so it can

help to have a different model to make forecasts for each step. 👎 **The downside is that training lots of models can be computationally expensive.**



- 3️⃣**Recursive strategy:** Train a single one-step model and use its forecasts to update the lag features for the next step. With the recursive method, we feed a model's 1-step forecast back into that same model to use as a lag feature for the next forecasting step. We only need to train one model, 👎**but since errors will propagate from step to step, forecasts can be inaccurate for long horizons.**



- 4️⃣**DirRec strategy:** A combination of the direct and recursive strategies: train a model for each step and use forecasts from previous steps as *new* lag features. Step by step, **each model gets an additional lag input**. Since each model always has an up-to-date set of lag features, the DirRec strategy can capture serial dependence better than Direct, 👎**but it can also suffer from error propagation like Recursive.**

